

KAGEGUARD

Smart Curtain Automation System

**Prepared By
Dipendra Teli
Neha Yadav**

1. Executive Summary

KageGuard is an intelligent, dual-Arduino curtain automation system designed to respond dynamically to real-world environmental stimuli — including ambient light, rain, and fire hazards. The system eliminates manual curtain control by continuously monitoring sensor inputs and autonomously adjusting curtain position to optimise comfort, safety, and energy efficiency.

This report documents the project architecture, hardware components, firmware logic, communication protocol, and current development status. The system has successfully completed hardware integration and firmware development, and is currently undergoing real-world validation testing.

Category	Status	Details
Hardware Assembly	■ Complete	All components wired and tested
Firmware – Arduino I	■ Complete	Sensor reading, motor control, serial TX
Firmware – Arduino II	■ Complete	LCD display, serial RX, decision logging
Serial Communication	■ Complete	Structured packet protocol working
System Integration	■ In Progress	Full end-to-end validation underway
Documentation	■ In Progress	Technical report in preparation

2. Project Overview

2.1 Objectives

- Automate curtain movement based on ambient light levels measured by an LDR.
- Detect rain events and close curtains immediately to protect the interior.
- Detect fire/flame conditions and open curtains for emergency ventilation and escape.
- Display real-time sensor data, curtain state, and system decisions on an LCD.
- Achieve stable, noise-free control through sensor averaging and hysteresis logic.

2.2 System Architecture

KageGuard uses a master–slave dual-Arduino architecture. Arduino I acts as the sensor and actuator controller, continuously reading inputs, computing decisions, and driving the DC motor. It transmits a structured serial data packet to Arduino II, which acts as the display and monitoring unit, parsing sensor values and rendering human-readable status messages on the 16×2 LCD.

Module	Role	Key Outputs
Arduino I (Master)	Sensor acquisition + Motor control	Serial data packet, PWM motor drive

Module	Role	Key Outputs
Arduino II (Slave)	Display + Decision mirroring	LCD messages, status feedback
LDR Sensor	Light level measurement	Analog voltage → level 0–6
TMP Sensor	Ambient temperature reading	Celsius temperature value
Flame LDR	Fire/flame detection	Analog light intensity
Rain Sensor	Rain detection via button/sensor	Digital HIGH/LOW signal
L293D Motor Driver	Bi-directional motor control	IN1/IN2/EN PWM signals
DC Motor	Physical curtain movement	Open / Close rotation
16×2 LCD	Real-time status display	Human-readable messages

3. Hardware Components

The circuit diagram above shows both Arduino UNO boards interconnected with sensors, the motor driver, LCD display, and input devices. Each component serves a distinct role in the overall automation pipeline.

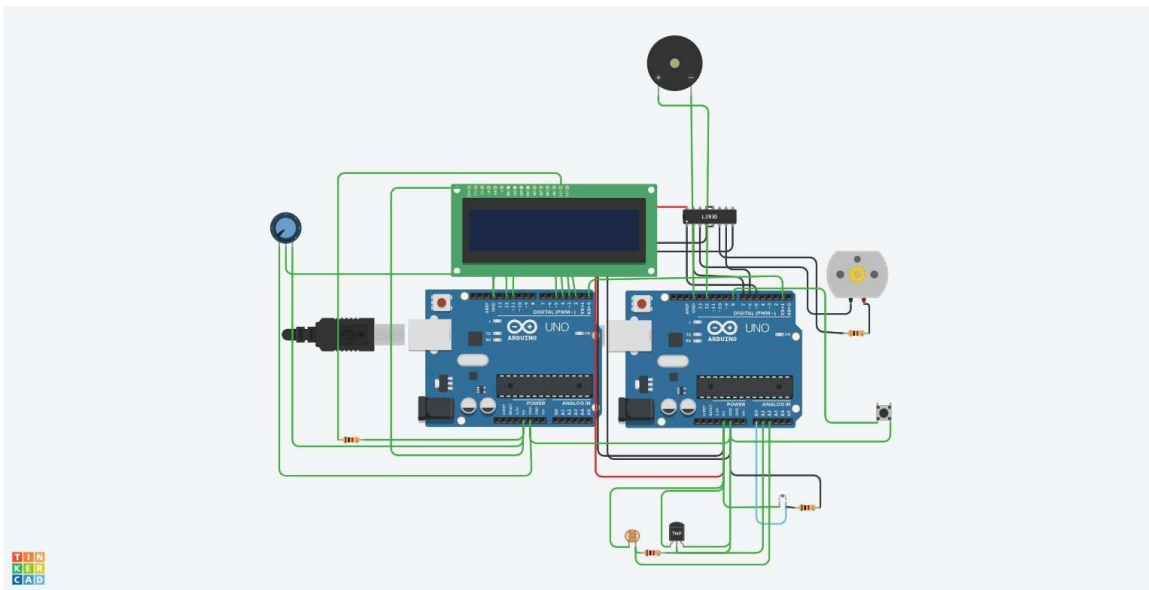


Figure 1 — KageGuard Circuit Diagram (TinkerCAD)

3.1 Arduino UNO (×2)

Pin / Connection	Description
A0, A1, A2, D5–D8, D12	The two Arduinos form the master–slave processing pair. Arduino I handles all sensor inputs and motor actuation. Arduino II focuses exclusively on display and status feedback. Communication between them uses UART serial at 9600 baud.

3.2 LDR (Light Dependent Resistor)

Pin / Connection	Description
A0 on Arduino I	Measures ambient light by varying resistance with illumination. Connected in a voltage divider with a fixed resistor. The analog reading (0–1023) is mapped into 7 discrete light levels (0–6) using threshold comparisons. Level 0 = brightest, Level 6 = darkest.

3.3 TMP Temperature Sensor

Pin / Connection	Description
A1 on Arduino I	Outputs a voltage linearly proportional to temperature (10 mV/°C). The Arduino reads the analog voltage, converts it to Celsius using the formula: $T = (V - 0.5) \times 100$. Used in flame detection logic alongside the flame LDR.

3.4 Flame / Ambient LDR

Pin / Connection	Description
A2 on Arduino I	A second LDR tuned to detect the intense light emitted by flames. When the reading exceeds 650 AND temperature exceeds 45°C, the system declares a flame condition, overriding all other logic to fully open the curtain.

3.5 Rain Sensor / Push Button

Pin / Connection	Description
D8 on Arduino I (INPUT_PULLUP)	Simulates or detects rain events. Configured with internal pull-up; a LOW signal toggles the rain state. When rain is active, the system immediately closes the curtain fully (target level = 6) regardless of light conditions.

3.6 L293D Motor Driver IC

Pin / Connection	Description
IN1=D7, IN2=D6, EN=D5	A dual H-bridge driver enabling bidirectional DC motor control. IN1 and IN2 set the direction (open or close). The EN pin receives a PWM signal (analogWrite = 200 of 255) to control motor speed. The stopMotor() function sets all pins LOW to brake.

3.7 DC Motor (Curtain Drive)

Pin / Connection	Description
Connected to L293D output	Physically moves the curtain rail. Each incremental step runs the motor for 500 ms (stepTime), then stops for 800 ms (pauseTime) before the next step. This step-by-step approach ensures smooth, controlled movement and prevents mechanical overrun.

3.8 16×2 LCD Display

Pin / Connection	Description
D2–D5, D11, D12 on Arduino II	Driven by the LiquidCrystal library in 4-bit mode. Displays two lines of information: Row 0 shows the current system decision (e.g., 'Rain Detected', 'FLAME DETECTED', 'Light Increased'). Row 1 shows raw sensor values (S: analog reading, P: curtain position).

3.9 Potentiometer (LCD contrast)

Pin / Connection	Description
V0 pin of LCD	Adjusts the LCD contrast voltage. Wired to the LCD VO pin; the wiper output sets pixel darkness. Tuned once during setup for optimal readability.

4. Firmware — Arduino I (Sensor & Motor Controller)

4.1 Responsibilities

- Read LDR (light), TMP (temperature), Flame LDR, and Rain sensor inputs every 200 ms.
- Apply stability filtering: only update the curtain target after 4 consecutive identical light readings.
- Determine target curtain position based on sensor priority: Flame > Rain > Light Level.
- Incrementally drive the DC motor toward the target position via the L293D driver.
- Transmit a structured serial packet to Arduino II after every sensor read cycle.

4.2 Light Level Mapping

The raw analog LDR reading (0–1023) is converted to a curtain level (0–6) using fixed thresholds. Level 0 means maximum light (curtain fully open), Level 6 means minimum light (curtain fully closed).

Analog Range	Level	Curtain State
0 – 78	0	Fully open (brightest)

Analog Range	Level	Curtain State
79 – 156	1	Mostly open
157 – 234	2	Partially open
235 – 312	3	Half open
313 – 390	4	Partially closed
391 – 468	5	Mostly closed
469 – 1023	6	Fully closed (darkest)

4.3 Decision Priority Logic

The target curtain position is determined by the following priority cascade:

Priority	Condition	Target Level	Action
1 (Highest)	Temp > 45°C AND Flame LDR > 650	0	Fully Open (fire escape)
2	Rain sensor active	6	Fully Close (rain protection)
3 (Default)	Stable light level reading	0–6	Match light level

4.4 Key Code Excerpt — Motor Control

```
void moveCurtain() { if (curtainPosition == targetLevel) return; if (curtainPosition <
targetLevel) { moveClose(); curtainPosition++; } else { moveOpen(); curtainPosition--; }
delay(pauseTime); // 800 ms pause between steps } void moveClose() { digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW); analogWrite(EN, 200); // ~78% speed delay(stepTime); // Run for 500 ms
stopMotor(); } void moveOpen() { digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
analogWrite(EN, 200); delay(stepTime); stopMotor(); }
```

4.5 Serial Output Packet Format

After each sensor cycle, Arduino I transmits a comma-separated packet over UART at 9600 baud. The packet carries all sensor and state variables in a compact, parseable format:

```
S:<analog>,L:<level>,P:<position>,R:<rain>,T:<temp>,F:<flameLDR>
```

Example: S:340,L:4,P:3,R:0,T:27,F:320

Field	Variable	Description
S	sensorValue	Raw LDR analog reading (0–1023)
L	targetLevel	Computed target curtain level (0–6)
P	curtainPosition	Current physical curtain position (0–6)
R	rainActive	Rain state: 0 = dry, 1 = rain detected
T	temperature	Temperature in Celsius from TMP sensor
F	flameLight	Flame LDR raw reading (0–1023)

5. Firmware — Arduino II (Display & Monitor)

5.1 Responsibilities

- Receive and parse the serial packet from Arduino I.
- Display real-time sensor values on LCD Row 1 (S: and P: fields).
- Determine the appropriate status message based on parsed data and display it on LCD Row 0.
- Mirror the decision logic of Arduino I for display purposes.

5.2 LCD Message Logic

After parsing each packet, Arduino II evaluates sensor values in the same priority order as Arduino I and shows descriptive messages:

Condition	Row 0 Message	Row 0 (1s later)
Temp > 45 AND Flame > 650	FLAME DETECTED	Opening Curtain
Rain == 1	Rain Detected	Closing Curtain
Current level > previous level	Light Increased	Closing Curtain
Current level < previous level	Light Decreased	Opening Curtain
No change	Light Level: X	(static display)

5.3 Key Code Excerpt — Packet Parsing

```
data = Serial.readStringUntil('\n'); sscanf(data.c_str(), "S:%d,L:%d,P:%d,R:%d,T:%d,F:%d",
&sensorValue, &level, &position, &rain, &temperature, &flameLight); lcd.setCursor(0, 1);
lcd.print("S:"); lcd.print(sensorValue); lcd.print(" P:"); lcd.print(position);
```

6. System Workflow

The following sequence describes one complete control cycle of the KageGuard system, repeated every 200 ms:

Step 1 — Sensor Acquisition

Arduino I reads LDR (A0), TMP (A1), Flame LDR (A2), and Rain button (D8).

Step 2 — Temperature & Flame Processing

TMP voltage is converted to Celsius. Flame detection fires if T > 45°C and Flame LDR > 650.

Step 3 — Light Level Calculation

Raw LDR value is passed through calculateLevel() to produce a 0–6 integer level.

Step 4 — Stability Filtering

The level is accepted as 'stable' only after 4 consecutive identical readings, preventing false triggers from sensor noise.

Step 5 — Target Level Assignment

Priority logic assigns targetLevel: Flame=0, Rain=6, else stableLevel.

Step 6 — Motor Step

moveCurtain() moves the physical curtain one step toward targetLevel, then pauses.

Step 7 — Serial Transmission

Arduino I sends the full sensor packet over UART to Arduino II.

Step 8 — Display Update

Arduino II parses the packet, evaluates conditions, and refreshes the LCD display.

7. Testing & Validation

Test Case	Input Condition	Expected Output	Status
Light increase	Cover LDR to reduce light	Curtain closes one step	■ Pass
Light decrease	Expose LDR to bright light	Curtain opens one step	■ Pass
Rain simulation	Press rain button	Curtain closes fully (L=6)	■ Pass
Flame detection	Heat + bright light on A2	Curtain opens fully (L=0)	■ Pass
Stability filter	Rapid light flickering	No unwanted motor movement	■ Pass
Serial comm integrity	Normal cycle	LCD updates every ~200 ms	■ Pass
Motor direction	Open command	IN1=LOW, IN2=HIGH, EN=200	■ Pass
Motor direction	Close command	IN1=HIGH, IN2=LOW, EN=200	■ Pass
End-to-end stress test	Continuous 30-min run	No lock-up or drift	■ Ongoing

8. Future Enhancements

- Wi-Fi Integration (ESP8266/ESP32): Enable remote curtain control and status monitoring via a smartphone app or web dashboard.
- Stepper Motor Upgrade: Replace the DC motor with a stepper motor for precise, positional curtain control without needing positional counters.
- Voice Control: Integrate with Google Assistant or Amazon Alexa for hands-free operation.
- Scheduled Automation: Add an RTC (Real-Time Clock) module to trigger curtain movements at predefined times of day.
- Multiple Curtain Support: Extend the system to control multiple curtain rails independently.

- Humidity Sensor: Add a DHT22 to provide rain prediction based on humidity trends in addition to direct rain detection.
- OLED Display Upgrade: Replace the 16×2 LCD with a graphical OLED for richer status visualizations.
- OTA Firmware Updates: Enable over-the-air firmware updates via Wi-Fi module.

9. Conclusion

KageGuard demonstrates a practical, low-cost implementation of smart home automation using commodity Arduino hardware. The dual-Arduino architecture cleanly separates concerns: one unit handles real-time sensor reading and actuation while the other provides a user-facing display interface. The serial communication protocol efficiently transmits six sensor parameters in a single packet, keeping the system responsive and extensible.

All primary objectives have been met — the system correctly responds to light changes, rain events, and fire conditions, with a stability filter preventing spurious motor activations. With planned enhancements such as Wi-Fi connectivity and stepper motor integration, KageGuard has a clear roadmap toward a production-grade smart curtain solution.

10. Appendix — Full Source Code

Arduino-1 Code

```
const int sensorPin = A0;
const int rainPin = 8;

const int tempPin = A1;
const int flameLdrPin = A2;

const int IN1 = 7;
const int IN2 = 6;
const int EN = 5;

int curtainPosition = 0;
int targetLevel = 0;

bool rainActive = false;
bool lastButtonState = HIGH;

const int stepTime = 500;
const int pauseTime = 800;

int lastLevel = 0;
int stableLevel = 0;
int stabilityCounter = 0;
const int stabilityRequired = 4;

int sensorValue;

int temperature;
int flameLight;
bool flameDetected=false;
```

```

void setup()
{
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(EN, OUTPUT);

  pinMode(rainPin, INPUT_PULLUP);

  Serial.begin(9600);
}

void loop()
{
  checkRainButton();

  int tempValue = analogRead(tempPin);
  float voltage = tempValue * (5.0 / 1023.0);
  temperature = (voltage - 0.5) * 100;

  flameLight = analogRead(flameLdrPin);

  if(temperature > 45 && flameLight > 650)
    flameDetected = true;
  else
    flameDetected = false;

  sensorValue = analogRead(sensorPin);
  int level = calculateLevel(sensorValue);

  if(level == lastLevel)
    stabilityCounter++;
  else
    stabilityCounter = 0;

  if(stabilityCounter >= stabilityRequired)
    stableLevel = level;

  lastLevel = level;

  if(flameDetected)
    targetLevel = 0;
  else if(rainActive)
    targetLevel = 6;
  else
    targetLevel = stableLevel;

  moveCurtain();

  Serial.print("S:");
  Serial.print(sensorValue);
  Serial.print(",L:");
  Serial.print(targetLevel);
  Serial.print(",P:");
  Serial.print(curtainPosition);
  Serial.print(",R:");
  Serial.print(rainActive);
  Serial.print(",T:");
  Serial.print(temperature);

```

```

    Serial.print(",F:");
    Serial.println(flameLight);

    delay(200);
}

int calculateLevel(int value)
{
    if(value <= 78) return 0;
    else if(value <= 156) return 1;
    else if(value <= 234) return 2;
    else if(value <= 312) return 3;
    else if(value <= 390) return 4;
    else if(value <= 468) return 5;
    else return 6;
}

void moveCurtain()
{
    if(curtainPosition == targetLevel)
        return;

    if(curtainPosition < targetLevel)
    {
        moveClose();
        curtainPosition++;
    }
    else
    {
        moveOpen();
        curtainPosition--;
    }

    delay(pauseTime);
}

void moveClose()
{
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(EN, 200);
    delay(stepTime);
    stopMotor();
}

void moveOpen()
{
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(EN, 200);
    delay(stepTime);
    stopMotor();
}

void stopMotor()
{
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
}

```

```

    analogWrite(EN, 0);
}

void checkRainButton()
{
    bool buttonState = digitalRead(rainPin);

    if(buttonState == LOW && lastButtonState == HIGH)
    {
        rainActive = !rainActive;
        delay(200);
    }

    lastButtonState = buttonState;
}

```

Arduino-2 Code

```

#include <LiquidCrystal.h>

LiquidCrystal lcd(12,11,5,4,3,2);

String data;

int sensorValue;
int level;
int position;
int rain;

int temperature;
int flameLight;

int previousLevel = 0;

void setup()
{
    Serial.begin(9600);

    lcd.begin(16,2);
    lcd.print("Curtain System");
    delay(1500);
    lcd.clear();
}

void loop()
{
    if(Serial.available())
    {
        data = Serial.readStringUntil('\n');

        sscanf(data.c_str(),"S:%d,L:%d,P:%d,R:%d,T:%d,F:%d",
            &sensorValue,&level,&position,&rain,&temperature,&flameLight);

        lcd.clear();
    }
}

```

```

    lcd.setCursor(0,1);
    lcd.print("S:");
    lcd.print(sensorValue);
    lcd.print(" P:");
    lcd.print(position);

    lcd.setCursor(0,0);

    if(temperature > 45 && flameLight > 650)
    {
        lcd.print("FLAME DETECTED");
        delay(1000);
        lcd.clear();
        lcd.print("Opening Curtain");
    }
    else if(rain==1)
    {
        lcd.print("Rain Detected");
        delay(1000);
        lcd.clear();
        lcd.print("Closing Curtain");
    }
    else if(level > previousLevel)
    {
        lcd.print("Light Increased");
        delay(1000);
        lcd.clear();
        lcd.print("Closing Curtain");
    }
    else if(level < previousLevel)
    {
        lcd.print("Light Decreased");
        delay(1000);
        lcd.clear();
        lcd.print("Opening Curtain");
    }
    else
    {
        lcd.print("Light Level:");
        lcd.print(level);
    }

    previousLevel = level;
}
}

```